Interfaces com usuário e Banco de dados

Nodify — Sistema de Acessibilidade Cefálica para Spotify

Seções: 3.3 e 3.4 **Plataforma:** Android · Kotlin **Persistência:** SQLite via Room 2.6.1

3.3 Interfaces com usuário

Implementação das interfaces de usuário conforme o projeto de interface, garantindo a funcionalidade e a usabilidade. Não precisa documentar esta parte, pois será visualizada no próprio software em execução.

Conforme a instrução, esta seção não é documentada: as interfaces serão demonstradas com o aplicativo em execução. Abaixo apenas a relação das telas implementadas, para referência durante a apresentação.

Tela	Função	O que será demonstrado
Login	Autenticação OAuth 2.0 com PKCE	Conectar a conta sem que a senha passe pelo aplicativo; retorno pelo *deep link*.
Início	Navegação principal	Estado da conexão, faixa em reprodução e acesso às quatro funções.
Calibração	Medir a amplitude do usuário	Sete passos guiados, com avisos de iluminação, enquadramento e rosto único.
Configurar Gestos	Mapear gesto para ação	Lista dos nove gestos, exclusividade de ação e indicador de progresso.
Rastreamento Ativo	Controle da reprodução	Gesto reconhecido virando comando no Spotify, com retorno visual.
Biblioteca	Escolher o que tocar	Navegação por playlists e faixas usando apenas movimentos da cabeça.

3.4 Banco de dados

Criação e configuração do banco de dados conforme o projeto de banco de dados, incluindo a inserção de dados iniciais e scripts de migração.

3.4.1 Criação e configuração

Item	Implementação
SGBD	SQLite, embarcado no próprio dispositivo
Camada de acesso	Room 2.6.1 — gera o código de acesso a partir de anotações e valida as consultas em tempo de compilação
Arquivo do banco	`sac.db`, no armazenamento privado do aplicativo
Versão do esquema	1
Esquema versionado	`app/schemas/com.sac.acessibilidade.data.NodifyDatabase/1.json`, exportado automaticamente a cada compilação
Declaração	`NodifyDatabase`, anotada com `@Database(entities = [GestureMappingEntity::class], version = 1, exportSchema = true)`
Instanciação	Fornecida como *singleton* pelo Hilt em `DataModule`

3.4.2 Modelo implementado

O modelo tem uma única entidade. O mapeamento entre gesto e ação é o dado que precisa sobreviver entre execuções e ser consultado a cada comando — é ele que garante a regra de que nenhuma associação gesto→ação existe fixa no código.

Tabela `gesture_mappings`

Campo	Tipo	Restrição	Descrição
`gesture`	TEXT	PK, NOT NULL	Gesto do vocabulário (`TILT_HEAD_RIGHT`, `NOD`…). Como chave primária, garante que um gesto tenha no máximo uma ação.
`action`	TEXT	aceita NULL	Ação associada (`NEXT_TRACK`…). Valor nulo representa "sem ação", permitindo desligar um gesto que o usuário não consegue executar.

DDL gerado pelo Room

```
CREATE TABLE IF NOT EXISTS `gesture_mappings` (
    `gesture` TEXT NOT NULL,
    `action`  TEXT,
    PRIMARY KEY(`gesture`)
)
```

Não há chaves estrangeiras nem índices adicionais: a tabela tem no máximo nove linhas, uma por gesto do vocabulário, e é sempre lida por inteiro ou pela chave primária. Um índice secundário não traria ganho de leitura e encareceria a escrita.

Operações disponíveis

Operação	Tipo	Finalidade
`observeAll()`	Consulta reativa (`Flow`)	Lê todas as linhas e reemite a cada alteração — é o que faz a tela refletir a mudança sem recarregar.
`upsertAll(mappings)`	Inserção e atualização	Grava o mapeamento. Como `gesture` é chave primária, regravar substitui em vez de duplicar.
`clearAll()`	Exclusão	Esvazia a tabela. Usada por "Restaurar padrões", sempre antes de regravar.

3.4.3 Inserção de dados iniciais

O aplicativo não usa *script* de carga. Na primeira leitura, o repositório detecta a tabela vazia e entrega os nove gestos do vocabulário, todos **sem ação associada**. A gravação efetiva no banco acontece na primeira vez que o usuário salva a configuração.

Gesto	Ação inicial
Inclinar para Direita / Esquerda	(sem ação)
Inclinar para Cima / Baixo	(sem ação)
Virar para Direita / Esquerda	(sem ação)
Aceno (sim)	(sem ação)
Piscar Olho Direito / Esquerdo	(sem ação)

> **Por que o estado inicial é vazio.** Um mapeamento de fábrica presumiria quais movimentos a pessoa consegue executar — exatamente o que um aplicativo de acessibilidade não pode presumir. Um usuário incapaz de virar o rosto para a esquerda receberia um comando inútil nesse gesto e teria de descobrir sozinho como trocá-lo. Começar vazio faz com que a configuração seja percorrida uma vez, e dela se saia com um mapeamento que funciona para aquela pessoa. A tela de configuração mostra o progresso, e a tela de rastreamento avisa enquanto nenhum gesto estiver associado.

3.4.4 Scripts de migração

O esquema está na **versão 1** e não sofreu alteração desde sua criação, de modo que **nenhuma migração foi necessária até o momento**. O projeto está preparado para quando isso mudar: o esquema é exportado e versionado em `app/schemas/` a cada compilação, o que dá ao Room o ponto de partida para gerar e validar a migração seguinte.

> **Limitação declarada.** A configuração atual usa `fallbackToDestructiveMigration()`: se a versão do esquema subir sem uma migração correspondente, o Room recria o banco e **os mapeamentos do usuário são perdidos**. É aceitável enquanto o esquema não evolui, mas precisa ser substituído por uma migração explícita antes da primeira alteração de estrutura. Fica registrado como pendência.

3.4.5 Persistência complementar

Nem todo dado persistido está em tabela. Dois conjuntos usam mecanismos próprios, por exigência técnica:

Dado	Mecanismo	Motivo
Parâmetros de calibração (11 valores)	SharedPreferences `sac_calibration`	Conjunto fixo de valores simples, sem relacionamento nem consulta — uma tabela não traria benefício.
Tokens de acesso ao Spotify	EncryptedSharedPreferences `spotify_tokens`	Exigem cifragem apoiada no Android Keystore, que o SQLite não oferece.

3.4.6 Evidências

A persistência foi verificada por **33 testes executados em aparelho real** (Samsung Galaxy S25+, Android 16), usando as classes reais — banco em memória para isolar cada caso, e as preferências do próprio aparelho. Todos passaram.

Conjunto	Testes	O que cobre
`GestureMappingDaoTest`	**6**	Banco novo vazio; gravar e ler; regravar substitui sem duplicar; ação nula sobrevive; limpar apaga tudo; o fluxo reemite após escrita.
`GestureMappingRepositoryIntegrationTest`	**7**	Primeiro uso; customização persiste e é relida; gesto sem ação vira linha com nulo; nova instância lê o mesmo banco; restaurar padrões não deixa linha órfã.
`CalibrationRepositoryImplTest`	**8**	Round-trip dos onze limiares sem troca de chaves; polaridade negativa preservada; recalibrar sobrescreve; nova instância lê o que foi gravado.
`SpotifyTokenStoreTest`	**12**	Cifragem real via Keystore; validade do token; limpeza completa; *state* do OAuth de uso único (proteção anti-replay).

```
./gradlew connectedDebugAndroidTest
33 testes de persistência — 0 falhas
```

3.4.7 Onde encontrar no repositório

Artefato	Caminho
Declaração do banco	`data/NodifyDatabase.kt`
Entidade (estrutura da tabela)	`data/gesture/GestureMappingEntity.kt`
Operações	`data/gesture/GestureMappingDao.kt`
Dados iniciais	`data/gesture/GestureMappingRepositoryImpl.kt`
Configuração e instanciação	`data/di/DataModule.kt`
Esquema versionado (DDL)	`app/schemas/com.sac.acessibilidade.data.NodifyDatabase/1.json`
Testes	`app/src/androidTest/java/com/sac/acessibilidade/data/`

Raiz dos caminhos de código: `app/src/main/java/com/sac/acessibilidade/`